Learning.AI

System Architecture, Feature Directory & Senior Technical Roadmap

Version 1.0 | Senior Cloud Solutions Architect Review | Production Readiness Blueprint

1. Executive Summary & Production Topology

Learning.AI is an AI-powered education and adaptive assessment platform designed for dynamic skill evaluation, real-time proctoring, and personalized learning paths. The system combines an adaptive test engine, biometric verification, WebRTC webcam proctoring, and comprehensive progress tracking.

> **Key Milestone Achieved (October 2026):**
> - Mongoose serverless cold-start buffering timeout (10000ms) permanently resolved.
> - Middleware reordered: connectDB guarantees MongoDB readyState === 1 before session/passport.
> - Live production verified: /api/health/db (200 OK) and /api/auth/login responding cleanly.

Active Deployment Topology:

Tier / Layer	Technology Stack	Live Production URL / Identifier
Frontend Client	React 18, Vite, Tailwind CSS	https://learning-ai-rswf.vercel.app
Backend API	Node.js 22, Express 4, Mongoose 7	https://learning-ai-mocha.vercel.app
Database Cluster	MongoDB Atlas (Replica Set)	learning-ai (Cluster: ac-thajia8)

2. Platform Capabilities & Functional Subsystems

Adaptive Exam Engine:

Dynamically adjusts test difficulty based on real-time student performance vectors. Implemented in AdaptiveDifficultyEngine.js and AdaptiveExamSession.jsx with real-time score calibration.

Biometric Verification & Camera Proctoring:

Enforces candidate integrity via facial snapshots and continuous webcam recording. Includes AdminBiometricApproval.jsx and real-time candidate feed monitoring in CameraMonitor.jsx.

Curriculum Roadmaps & Gamification:

Structured learning tracks (Frontend, Backend, AI/ML, Mobile, DevOps) mapped to granular lesson modules. Integrated with XP points, streak counters, and category mastery levels (xp.service.js).

Personalized Content Generation:

Generates custom question sets (PersonalizedQuestionSetService.js) targeting individual weak spots identified from prior assessment results and quiz history.

3. Senior Architectural Evaluation & Gap Analysis

A rigorous review of the codebase identifies five specific areas required to achieve full production excellence:

Architecture Finding	Severity	Current State & Senior Architect Remedy
1. Video Storage on Ephemeral Disk	**CRITICAL**	recordingService.js writes to local /uploads on disk. On Vercel, serverless disks are read-only and ephemeral. Migrate to S3/R2 direct uploads.
2. Missing Database Seed Data	**HIGH**	Collections like "roadmaps" are empty in Atlas, causing /api/roadmaps/frontend to 404. Deploy an automated seed script for all core roadmap tracks.
3. Tailwind CDN in Production	**HIGH**	index.html loads cdn.tailwindcss.com. Browser warns not for production. Configure Tailwind via PostCSS inside Vite for optimal performance.
4. Pure Stateless JWT Auth	**MEDIUM**	Passport and session store add cold-start overhead. Transition Google OAuth and protected routes to pure JWTs (eliminate MongoStore dependency).
5. Rate Limiter Tuning	**MEDIUM**	Global 100 req / 15 min rate limiter risks dropping high-frequency proctoring telemetry chunks. Segment limits by endpoint purpose.

Detailed Technical Analysis:

1. Video Storage & Proctoring Architecture

In cameraRoutes.js and recordingService.js, video chunks sent from candidate webcams are currently appended to local files using fs.appendFile. On Vercel, each request may execute on a separate lambda instance with a clean ephemeral file container. The required architectural pattern is presigned cloud object URLs (Amazon S3 or Cloudflare R2), allowing candidate browsers to stream WebM video chunks straight to cloud buckets.

2. Seeding & Database Integrity

When the frontend requests /api/roadmaps/frontend, the route queries Roadmap.findOne({ roadmapId: ... }). Because the database does not contain seed documents, the endpoint returns 404. Incorporating an automated idempotent seeder ensures default tracks (frontend, backend, full-stack, ai-ml) are populated upon boot.

3. Frontend Bundle & PostCSS Optimization

Tailwind CSS loaded through cdn.tailwindcss.com evaluates all style rules on client-side JS runtime. Transitioning to Vite PostCSS build plugin bundles purged CSS, eliminating runtime evaluation and latency.

4. Phased Production Blueprint & Technical Roadmap

Phase 1: Immediate Production Hardening (Priority 1)

- Implement S3/R2 presigned URL generation for proctoring video chunk streaming.
- Run database seed script for roadmaps and question banks in MongoDB Atlas.
- Migrate Tailwind CSS from CDN to PostCSS/Vite inside app/frontend.
- Remove dead/duplicate backup files in frontend src/components to reduce bundle footprint.

Phase 2: Architectural Decoupling & Scalability (Priority 2)

- Refactor Google OAuth to stateless JWT exchange (session: false), deprecating MongoStore.
- Segment express-rate-limit: auth (10/min), general API (120/min), proctoring telemetry (600/min).
- Create compound indexes in Atlas for examId, sessionId, learnerId, and status.
- Add React Error Boundaries around AdaptiveExamSession and Roadmap modules.

Phase 3: Enterprise Observability & Operations (Priority 3)

- Setup Sentry error monitoring across frontend SPA and Vercel serverless functions.
- Implement automated end-to-end testing with Playwright for login, exam, and proctoring flows.
- Consolidate custom domains under unified root (e.g., app.learningai.com & api.learningai.com).

5. Architectural Sign-off & Recommendation

With the resolution of the Mongoose connection lifecycle and query buffering bottleneck, the core infrastructure now operates reliably on Vercel Serverless. By systematically addressing the cloud storage and database seeding milestones outlined above, Learning.AI will represent a benchmark educational platform capable of supporting thousands of concurrent examination sessions with complete integrity.

Approved by: Senior Full-Stack Cloud Solutions Architect

Date: October 18, 2026 | Document Status: ACTIVE & APPROVED | Learning.AI v1.0